CIS 3250: Software Design lll
Dr. John Akinyemi
September 23, 2026
Due: Sep 30, presentation due oct 6

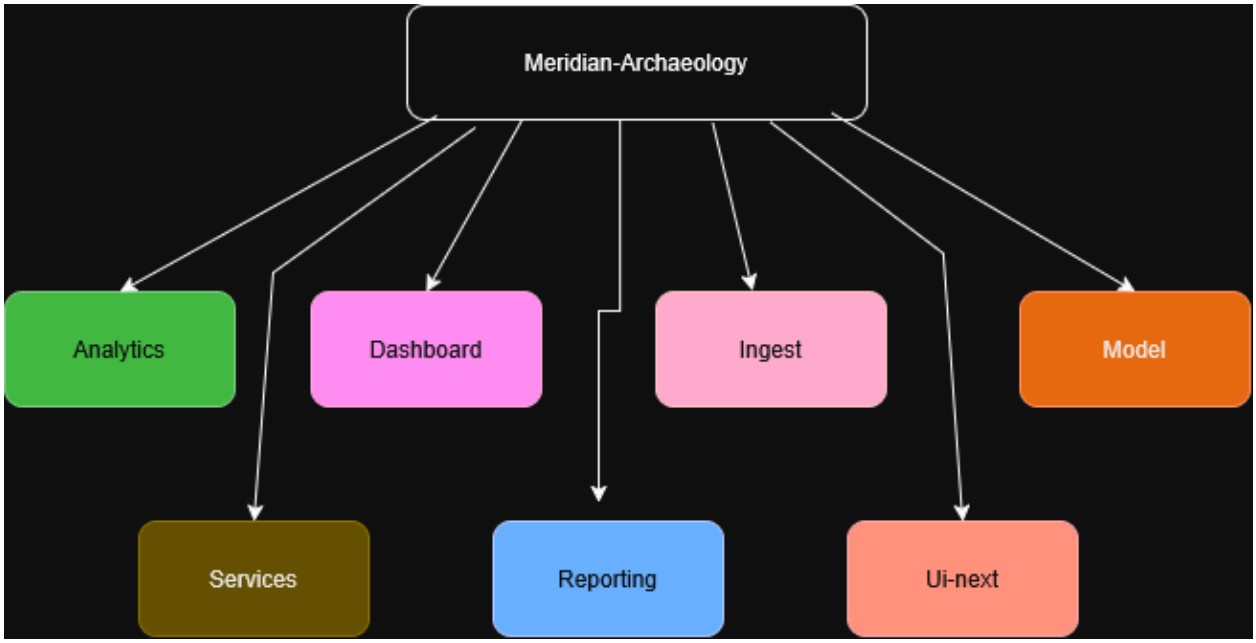
Team: Almanac

Group Members:

- Addison Broschell
- Arianna Wallace
- Harveen Harveen
- Leo Goebel
- Aithan McIntyre

1. System Map (all)
2. End-to-end trace (harveen)
3. Behaviour of Subsystem (arianna)
4. Subsystem Biography (addison)
5. Open Tickets in Subsystem (leo)
6. Here be Dragons in Subsystem (aithan)
7. Agent Appendix

1. System Map



Analytics: Python, this subsystem is responsible for importing, exporting and summarizing data and exposes SQLite database and model files.

Dashboard: Javascript, CSS and HTML. This subsystem is responsible for displaying the charts, tables and how the program looks as a whole. Dashboard exposes a web interface using API requests.
Ingest: C, this subsystem is responsible for station constants, tail handling and accessing the store. This exposes binary telemetry frames.
Model: Fortran, this subsystem is responsible for simulating crop growth and soil water balance to generate forecasts. It exposes fixed width input deck and output files.
Reporting: Python, this subsystem is responsible for formatting, generating and testing reports. Reports and export files are exposed.
Services: Java, this subsystem is responsible for the admin tool, in memory cache and api. Services expose HTTP REST API (JSON) interface.
Ui-next: TypeScript/React, this subsystem is responsible for the navigation, layout and screens for the program. Ui also exposes web interface and API requests.

DATA FLOW: Data moves through the system as weather readings are collected, stored, processed and displayed. A binary telemetry frame is stored as a SQLite row, converted into a fixed-width deck for the FORTRAN model, transformed into forecast records, and finally sent as a JSON to the Dashboard display.

CONTROL FLOW: Control is managed by the Analytics Orchestrator, which coordinates reading weather data, running the FORTRAN model, and storing forecast results. When a user requests information from the Dashboard, the API retrieves the required forecast data from the database and returns it to the interface.

1. README.md Lie
   README claim: the reporting subsystem replaced the cobol RPTGEN writer in 1997.

   Reality: The git history shows the reporting subsystem being introduced in 2017 in commit a3298ea, whose commit message specifically says it replaces the cobol RPTGEN writer. Therefore, the READMEs 1997 date does not match the repository history.

   a3298ea | 2017-08-07 | R. Halloran | Batch report writer (replaces the COBOL RPTGEN); season + station reports

2. The README says that Meridian is a three-tier application which includes: ingest, services, and dashboard. But if we look in the repository there are seven subsystems: analytics, dashboard, docs, ingest, model, reporting, and services. This is a contradiction because the current repository is not organized into only three parts as described in the README. The current system is divided into seven subsystems rather than the three that are outlined in the README.

3. The night batch: In the README it states that the nightly batch is scheduled using Quartz and tells the developers to look at services/src/main/resources/quartz.properties. The services directory does not contain the quartz.properties file that is stated in the README. The services directory contains Java source files instead. This is a contradiction because if a developer was following the README and was looking for the quartz.properties it would not exist.

2. End-to-end trace (Whole System)

A weather reading starts at a field station and is sent as a binary telemetry frame to the telemetry ingest subsystem. The ingest system validates the frame and stores the data in the SQLite database as a database row.

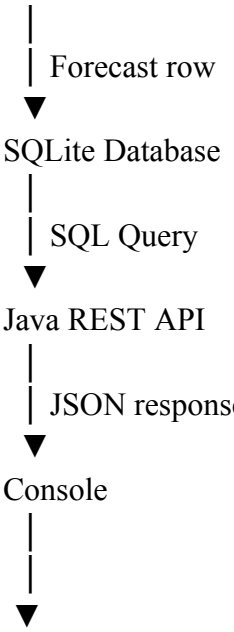
The Analytics Orchestrator reads the stored weather data from the database and creates a fixed-width deck file for the FORTRAN model. The FORTRAN model processes this input and produces a fixed format output file. The parser reads the model output and stores the forecast results back into the database.

The Java API retrieves the forecast data from the database and returns it as a JSON response. Finally, the Console requests the data from the API and displays the forecast value or chart to the user.

DATA FLOW:

Field Station
  │
  │ Binary telemetry frame
  ▼
Telemetry Ingest (C)
  │
  │ SQLite row
  ▼
SQLite database
  │
  │ weather observations
  ▼
Analytics Orchestrator (Python)
  │
  │ Fixed - width deck file
  ▼
FORTRAN Model
  │
  │ FIxed - format output file
  ▼

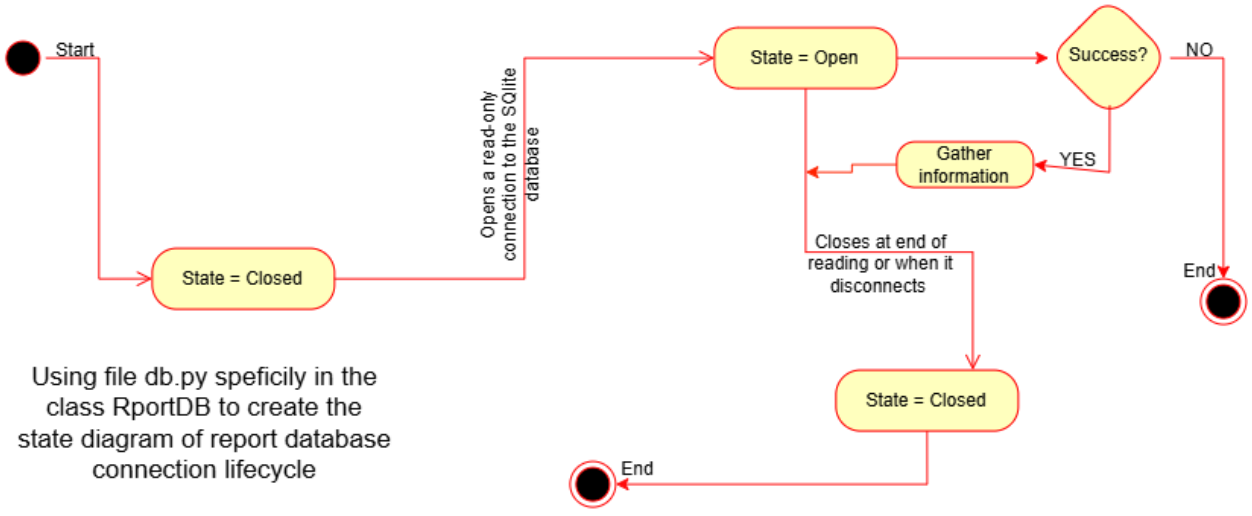
Parser (Python)



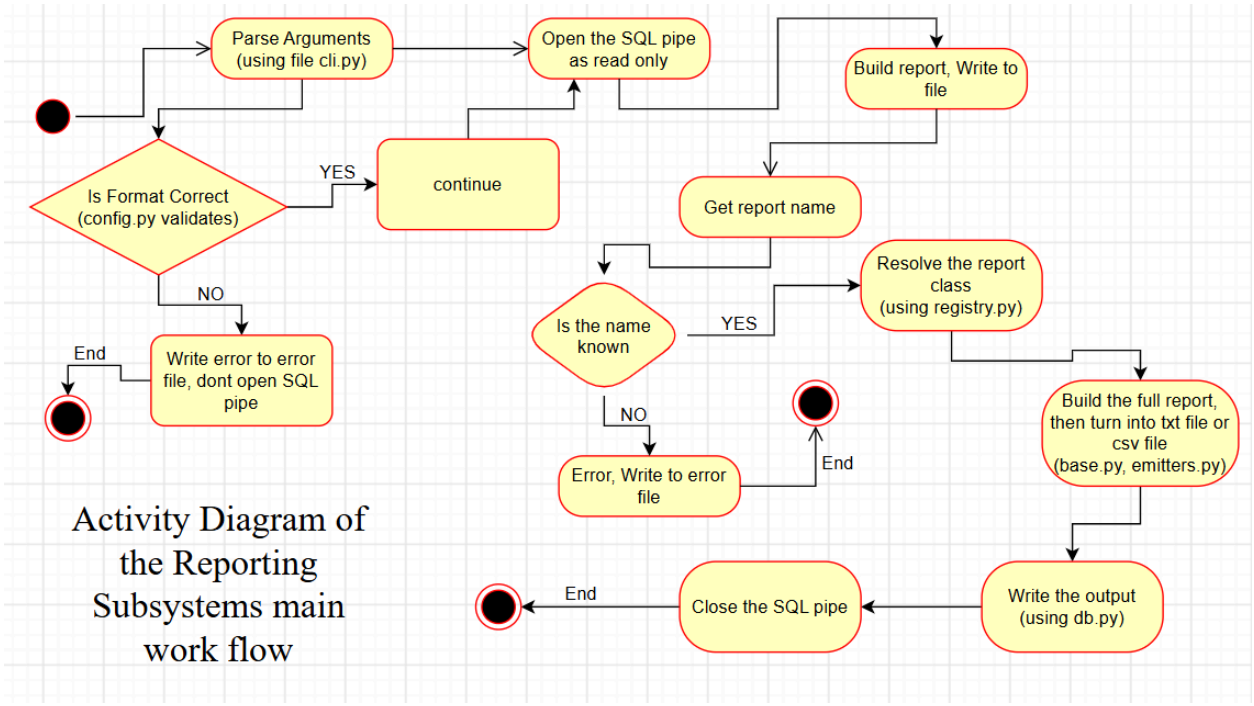
This follows the required path:
Frame → ingest → store → orchestrator → model → output → parser → store → API → console

3. Behaviour of Subsystem

State Diagram:



Action Diagram:



Reporting Interface Contract:
The inputs that the file in the reporting system needs to work varies. season_summary.py requires a year and a valid forecast, station_report.py needs a station and also a valid forecast, daily_detail.py requireIs both a station and a year, anomaly.py needs nothing mandatory though you can provide optional station and year, and coverage.py needs a forecast.

For the output: the reporting system writes all errors to the directed python error file, it also returns and writes reports in either a csv format or a text file format.

Invariants: The database is ever read-only. It does not create, alter or write tables. Everything has to be laid out according to the config.py. There are fixed table and column widths and fixed numerical fields.

Error behaviour: Errors are dealt by recording the error in the error file “std.err” that python uses. They also return exit code 1 when the program does not behave as expected. For database errors db.py raises “ReportDBError” when the store is missing, cant be opened, a query fails, or is accessed after the connection has been closed, it reports it and exits with error code one.

Exact format and Dependencies: db.py expects reading, forecast, and daily forecast tables. config.py needs page widths of 80 or 132 columns, and needs it to be formatted according numfmt.py which specifies numeric values are right-aligned, values too wide become asterisks, missing values appear as “.” , and clipped text gets a trailing “>”.

4. The subsystem's biography (git archaeology)

The reporting/export subsystem originally came to be in 2017. The first commit (a3298ea1f44981baf0886dda0a833989d63c60e4) was made by R.Halloran on August 7, 2017 and is titled: “Batch report writer (replaces the COBOL RPTGEN); season + station reports.” This commit created the main reporting files and added the season summary, station report, and daily detail reports. The reports use BaseReport, and the reports are connected to the command-line interface through the report registry. This shows that the subsystem was created as a replacement for the older COBOL reporting process while keeping the existing reporting functionality. In 2021, P. Adeyemi updated the reporting subsystem. In commit (8cce53b3232a08be1036f6b06e6b9c3bb70e3e6f), the column layout width was updated to 132 columns. In 2023, the reporting subsystem was expanded again. In commit (ec164274f86c435794a0e43663aff0e083b9d496), which was made by P. Adeyemi on August 28, 2023, coverage and anomaly reports were added, folded query helpers were also added along with tests. This shows that the subsystem had grown beyond the original season, station, and daily reports. While looking at the 2017 commit, I found this piece of code in the DailyDetailReport that describes why the system has two different column layouts:

```
def _columns(self) -> ColumnSet:
    if self.config.is_wide():
        # Wide fan-fold: generous fields, biomass gets 9 so a heavy
        # crop year does not overflow.
        return ColumnSet(
            [
                Column("doy", "DOY", 4, kind="int"),
                Column("sw", "SoilWat", 9, kind="float", decimals=2,
                       unit="mm"),
                Column("et", "ET", 9, kind="float", decimals=3, unit="mm"),
                Column("drain", "Drain", 9, kind="float", decimals=3,
                       unit="mm"),
                Column("biom", "Biomass", 10, kind="float", decimals=2,
                       unit="g/m2"),
                Column("lai", "LAI", 8, kind="float", decimals=3,
                       unit="m2/m2"),
            ]
        )
    # Narrow: tight fields; matches the historical MERIDIAN.OUT widths
    # (F7.x) closely so a printout lines up with an archived model run.
    return ColumnSet(
        [
            Column("doy", "DOY", 4, kind="int"),
            Column("sw", "SoilWat", 8, kind="float", decimals=2,
                   unit="mm"),
            Column("et", "ET", 8, kind="float", decimals=3, unit="mm"),
            Column("drain", "Drain", 8, kind="float", decimals=3,
                   unit="mm"),
            Column("biom", "Biomass", 8, kind="float", decimals=1,
                   unit="g/m2"),
            Column("lai", "LAI", 7, kind="float", decimals=3,
                   unit="m2/m2"),
        ]
    )
```

This code is load bearing because it uses two different column layouts. The narrow column layout was designed to match the historical MERIDIAN.OUT format. The wide layout creates extra space for the 132-column output (stated in the 2021 commit). “Both widths are still supported because a lot of the ministry's terminals never got wider than 80.” (2017 Commit). This is why the older, narrow layout has not simply been removed. It exists for compatibility with older terminals and historical output requirements.

5. Open ticket:

MRD-238	2025-11-14	reporting	major	The report writer selects the current forecast per station with MAX(run_at) compared as text. A manual backfill written in a different timestamp format sorts wrong, so a report can print an older run as the one "in force".

Issue location: db.py line 211 within def latest_forecast_for_station.

Root cause: The string being passed to _one extends to other lines that should be run as code.

Fix would touch: _standing_block (affects the Station report) and test_latest_forecast_uniform_timestamps (_forcast converts the row). BaseReport.render emits StationReport as text.

6. **Here Be Dragons**

1. Dragon: Fixed width reporting format

   Where: reporting/reporting/README.md, format/numfmt.py and format/columns.py

   Why it matters: it matters because the reporting system depends on the exact column widths so current reports line up with older archived reports. Changing spacing, widths or alignment would shift the entire report layout. Numbers that are too large are even replaced with asterricks rather than being allowed to change the column width, showing how strict the format is.

   What a new developer should know: Do not change column widths, spacing or alignment, or overflow behaviour without checking how it affects the full report format.

2. Dragon: Latest forecast depends on timestamp text formatting

   Where: reporting/reporting/db.py, latest_forcast_for)station() and latest_run_at()
   (This is also documented in issue MRD-238.)

   Why it matters: The system uses MAX(run_at) to find the newest forecast, but run_at is stored and compared as text, if timestamps are entered using different formats, the text sorting can be wrong and an older forecast could be reported as the newest one.

What each developer should know: Any code that writes or changes run_at values needs to keep the timestamp format consistent. Before changing how timestamps are stored or compared, they should check an eerie part of the system that relies on the current format.

3. Dragon: Stations with no forecasts can disappear from the coverage report

   Where: reporting/reporting/db.py in all_stations_ids() and reporting/reporting/reports/coverage.py

   Why it matters: The coverage report builds its station list from stations already found in the forecast table. A station with no forecasts at all can therefore be left out instead of being shown as missing.
   What a new developer should know: The station list used by the coverage report is not necessarily the complete list of expected stations. A developer should check the canonical station list before assuming that the stations returned from the database represent every station that should exist.

7. Agent Appendix

How you will declare agent use:

- At the bottom of every page / deliverable a statement will be included stating that no AI was used or declaring the use of Agent and how it was used.
- If we are declaring that an Agent was used, we will be very clear and specific to highlight where and how we used it.
- Additionally, we will include what prompt the AI was given, what we used and what we did not.

In question 2, AI was used to explore the meridian codebase and identify useful investigation commands. For example, the AI suggested using tree -L 2 to examine the repository structure and find ingest analytics services dashboard -type f | head -100 to locate important source files. The results were verified by running the commands and using the output to identify subsystem directories, source files and relationship between components.

In question 3, AI was used to help find the files needed for a state diagram, for the activity diagram, and explanation of what a real interface contract is, examples of Invariants, error behavior, and exact formats and dependencies. (only what i used will be included)
Prompts given:

1. dont edit code, look in the reporting/reporting folder, give me files that can be made into A state diagram of a stateful element in your subsystem (for example: a telemetry frame

moving through accepted / rejected / suspect; a request through its lifecycle; the crop through its growth stages)

2. dont edit code, look in the reporting/reporting folder, give me files that can be made into An activity diagram of its main workflow (for example: the nightly batch; a request handler; the deck-write → run → parse cycle)
3. explain what a real interface contract () inputs, outputs, invariants, error behaviour, and any exact formats it

depends on. Be precise enough that another team could call your subsystem correctly from your document alone) is

4. dont edit code, look in the reporting/reporting folder, give me files that can be made into Its real interface contract: inputs, outputs, invariants, error behaviour, and any exact formats it

depends on. Be precise enough that another team could call your subsystem correctly from your document alone.

5. with the reporting folder give me examples of invarients
6. with the reporting folder give me examples of error behavior and exact formats and dependencies

What i Used:

1. The strongest state-diagram candidate is the report database connection lifecycle: db.py defines ReportDB: it opens a read-only connection, allows queries while open, and closes it explicitly or when its context manager exits. A diagram could show Closed → Open → Closed, with an open-failure path and a query-after-close error.
2. For an activity diagram of the reporting subsystem's main workflow, start with these files: cli.py: the main control flow in main() and run_report(), including argument parsing, early exits, error handling, and output. config.py: configuration creation and validate(), which provides the validation decision and invalid-input path. registry.py: maps the requested report name to its report class; an unknown name raises an error. base.py: shared report work: build the document and blocks, then choose text or CSV rendering. db.py: opens and closes the read-only database and provides report data. emitters.py: turns the document into text or CSV and writes it to stdout or a file. For the report-specific part of the workflow, add one concrete report module, such as daily_detail.py. For the text-pagination branch, add layout.py. The diagram's core path is: parse arguments → handle --list or missing report → build and validate config → resolve report class → open database → build report document and query data → render as text or CSV → write output → close database. Include branches for invalid config, unknown report, report/database errors, and text pagination.
3. A real interface contract is the precise description of how another team can interact with a subsystem without needing to inspect its implementation or guess what it expects.

It answers five practical questions: Inputs: What can callers provide? Which fields are required, optional, or restricted? Outputs: What does the subsystem return or write, and in what form? Invariants: What must always be true, such as ordering, normalization, read-only behavior, or stable identifiers? Error behavior: What happens for invalid input, missing data, unavailable dependencies, or failed operations? Does it raise an exception, return an error value, or use an exit code? Exact formats and dependencies: What schemas, encodings, field widths, units, protocols, or external conventions must callers and downstream systems follow?

4. Per-Report Caller Requirements season_summary.py: requires a year; errors if no forecasts exist for it. station_report.py: requires a station; errors if it has no forecasts. daily_detail.py: requires a station and year; errors if no daily series exists. anomaly.py: station and year are optional filters; no matches produce an explanatory empty-detail section. coverage.py: no station/year input required; errors if there are no forecasts.
5. Database problems: db.py raises ReportDBError when the store is missing, cannot be opened, a query fails, or a query is attempted after closing the connection. The CLI reports these as database errors and returns exit code 1. Database dependency: db.py expects an existing SQLite store containing reading, forecast, and forecast_daily tables. It opens the database read-only and only queries it.
6. The report database is read-only. ReportDB opens the SQLite store in read-only mode and report queries use SELECT; reporting must not create, alter, or write tables. See db.py. Only supported layout settings are valid: page width is 80 or 132, and format is text or csv. A validated ReportConfig must also have a year in 1900..2100 when supplied, and a nonblank station ID. See config.py.

In question 4, AI was used to help explain the commit (a3298ea1f44981baf0886dda0a833989d63c60e4) in depth so that I could get a better understanding of the comments and the actual functionality of the code.
Given Prompt to Gemini: “Can you explain this commit in more depth so I can understand what is happening more clearly. Then can you summarize it”
The response: This commit introduces daily_detail.py, a Python report module that replaces a legacy 1996 COBOL report generator (RPTGEN).

- **Purpose:** Queries daily crop forecast data (forecast_daily) for a given station and year, outputting six key agricultural metrics (Soil Water, ET, Drainage, Biomass, LAI, and DOY).
- **Layout & Display:** Supports both traditional 80-column legacy terminal views and expanded 132-column wide views. The wider layout prevents field overflow on high-yield crop years.
- **Safety:** If numbers exceed field width, it prints asterisks (****) rather than silently truncating data.

- **Structure:** Implements DailyDetailReport to generate three output blocks:
    1. **Header:** Station info, crop year, simulated day range, and expected yield.
    2. **Table:** Daily simulated model metrics row-by-row.
    3. **Summary:** End-of-season roll-ups including total ET, total drainage, peak LAI, and peak biomass with their respective peak days.

What was used: I used a general summary to help answer the load bearing code archeology part of question 4.

In question 5, GPT 5.5 Luna found all uses of a function as it ran meridian. Traced where latest_forecast_for_station is used and then traced where the functions it was used in are called. I used the file and line locations to find where functions were being used and check what the AI got correct. All the traced files were correct and I did not use the explanation it gave.

Unless otherwise stated: no other AI help was used.